How Well Do Agent Failure Taxonomies Travel? A Cross-Dataset Empirical Analysis of Failure Labels in Public Agent Trajectory Datasets

Abhinav Tharamel Baiju[1,*], Tejaswini Viswanath[2]

[1]Independent Researcher, Chicago, Illinois, USA
[2]Independent Researcher, Chicago, Illinois, USA

[*]Corresponding author: Abhinav Tharamel Baiju, abhinavtb@gmail.com
Tejaswini Viswanath: tejaswiniviswanath@gmail.com

Abstract

The artificial intelligence (AI) agent-failure taxonomies examined here were developed using their authors' own trajectories, yet the field routinely treats these taxonomies as portable, universal descriptions of why agents fail. Whether failure categories mean the same thing or occur at comparable rates outside their home benchmark is essentially untested. We audit the portability assumption empirically: we map two independently produced public failure datasets to a single unified scheme and test whether the resulting failure-label distributions travel across datasets, domains, and models, while directly auditing the quality of the released labels. We annotate 384 agent failures, consisting of 200 AgentErrorBench trajectories (ALFWorld, GAIA, WebShop) and 184 Who&When multi-agent failures, into a 13-class unified coding scheme defined for this audit, under a prospective analysis plan with prespecified hypotheses (H1–H3). Who&When's free-text attributions were blind double-coded (agreement 69.6%, Cohen's $\kappa = 0.662$) with consensus adjudication; confirmatory tests use χ^2 tests of independence with Cramer's V and Monte Carlo permutation p-values. Harmonized label distributions differ dramatically across datasets ($\chi^2 = 160.48$, $df = 12$, $p = 4.2 \times 10^{-28}$, Cramer's $V = 0.646$; H1 supported), across domains ($p = 4.1 \times 10^{-23}$), and across models ($p = 0.002$, $V = 0.30$). H2 and H3 are not supported: coordination failures do not dominate the multi-agent data, and the empty-label rate is not established to exceed 15%. A direct label-quality audit finds 13.5% of AgentErrorBench annotations carry empty failure labels (concentrated in one model's trajectories), inconsistent category strings, and documentation mismatches. Because tasks, systems, and annotation protocols co-vary across the datasets, the gaps cannot isolate a taxonomy effect. Our results limit generalization of observed failure frequencies and show that label noise in public failure datasets is itself a first-order problem for the field. In these data, failure-label profiles are benchmark-, domain-, and model-relative. The distributional gaps limit generalization of the observed failure frequencies but cannot establish them on their own. We close with concrete recommendations for benchmark designers.

1 Introduction

The evaluation of large language model (LLM) agents has matured rapidly. Benchmarks now span embodied control (ALFWorld [9]), open-ended assistant tasks (GAIA [4]), web interaction

(WebShop [12], WebArena [17]), tool use (API-Bank [2]), software engineering (SWE-agent [11]), and general agentic competence (AgentBench [3]), and recent surveys systematize this fast-moving evaluation landscape [15]. In parallel, agent *architectures* have converged on a recognizable pattern: the reason–act loop popularized by ReAct [14], in which the model interleaves chain-of-thought reasoning with tool calls, wrapped in scaffolding for memory, planning and error handling.

Alongside the benchmarks, a parallel literature proposes *taxonomies* of agent failure: structured vocabularies (planning errors, tool errors, grounding errors, and so on) intended to explain *why* agents fail. These taxonomies increasingly serve as the shared language of the field: papers compare "planning failures" across systems, and debugging frameworks target taxonomy categories as if they were stable, well-defined kinds.

The taxonomies examined here were developed using their authors' own trajectories, in their own environments, with their own annotation protocols. Whether the resulting categories mean the same thing, or even occur at comparable rates, outside their home benchmark is essentially untested. If failure labels do not travel across datasets, then cross-paper comparisons built on them rest on sand, debugging tools tuned to one benchmark's failure profile may misfire on the next, and the growing practice of treating taxonomy categories as portable measurement instruments [15] is premature.

This paper presents a cross-dataset empirical analysis of that portability: to our knowledge, the first to map two independently produced public failure datasets to a single unified scheme and test the resulting distributions. We take two public, independently produced agent-failure datasets with incompatible native labeling schemes and ask four prespecified research questions:

- **RQ1.** What is the distribution of failure labels within each dataset?
- **RQ2.** After mapping to one unified scheme, which failure categories dominate *across* datasets, and which are dataset-specific?
- **RQ3.** Do failure labels mean the same thing across datasets, and how clean are the released labels themselves?
- **RQ4.** Do failure distributions differ by agent domain (embodied, web, general-assistant, multi-agent)?

We specified directional hypotheses in advance: (H1) label distributions differ significantly across datasets; (H2) planning failures form the plurality in web-navigation data while coordination failures dominate multi-agent data; (H3) at least 15% of native labels cannot be mapped without definitional guesswork. H1 is strongly supported; H2 and H3 are not, and the ways in which they fail are themselves informative. An additional exploratory analysis (not prespecified) examines whether failure profiles differ across the evaluated language models. We stress upfront that the two datasets differ simultaneously in tasks, systems, and annotation protocols, so distributional differences alone cannot isolate a taxonomy effect; we frame the study as a descriptive audit of harmonized failure annotations and report the confounds explicitly rather than claiming a controlled test of portability.

Our contributions are: (1) a cross-dataset quantitative analysis of agent failure labels (384 annotated failures, two independently produced datasets, four domains, three models, one unified

scheme); (2) evidence that harmonized failure-label distributions are strongly dataset-, domain- and model-dependent; (3) a direct audit documenting label noise (empty labels, inconsistent strings, documentation mismatches) in a published failure benchmark; (4) concrete recommendations for benchmark designers; and (5) fully public data, mappings and analysis code for reproduction.

1.1 Related work

Agent benchmarks. ALFWorld [9] aligns text-based policies with embodied execution; GAIA [4] poses 466 real-world assistant questions requiring reasoning, multimodality and tool use; WebShop [12] provides a large-scale simulated e-commerce environment for grounded web agents; WebArena [17] extends web evaluation to 812 long-horizon tasks in self-hosted environments; API-Bank [2] targets tool-augmented LLMs with 73 APIs and annotated dialogues; SWE-agent [11] reframes software engineering agents around the agent–computer interface; and AgentBench [3] evaluates 29 LLMs as agents across eight interactive environments. [15] survey the evaluation landscape. These benchmarks measure *task success*; they do not explain *why* agents fail, which is the role taxonomies claim.

Agent architectures. ReAct [14] established the reason–act trajectory structure that underlies most modern agents and, consequently, most failure trajectories: a failure is almost always a breakdown somewhere in an interleaved sequence of thoughts, actions and observations. Our unified scheme's classes (planning, tool selection, formatting, execution, observation reading) map directly onto positions in this loop, which is why the scheme can serve as a common vocabulary at all.

More benchmarks, more failure surfaces. The pattern (each benchmark measuring success while leaving failure causes unlabeled) holds beyond our two datasets. ALFRED [8] is the embodied instruction-following benchmark on which ALFWorld is built; TPTU [5] benchmarks task planning and tool usage; TravelPlanner [10] stresses multi-day travel planning with hard constraints; and τ-bench [13] evaluates tool–agent–user interaction in realistic domains. Each would benefit from a portable failure vocabulary, and each would likely exhibit the same dataset-specific label skew we document here, since their task structures foreground different failure classes.

Learning from failures. A parallel literature treats failure as training signal rather than as object of study. Reflexion [7] lets agents verbally reflect on failed trajectories and store the reflections as episodic memory, improving subsequent attempts without weight updates; its success depends entirely on correctly *diagnosing* why a trajectory failed. AgentPex [6] automatically detects failures in agentic traces, a step toward scalable annotation. Both lines of work presuppose what we test: that failure labels are reliable and comparable. Our label-noise audit suggests that presupposition is shaky.

Failure taxonomies and debugging. [18] introduced the AgentErrorTaxonomy (error types across memory, reflection, planning, action and system modules), the AgentErrorBench set of annotated failure trajectories from ALFWorld, GAIA and WebShop, and the AgentDebug three-stage debugging framework. [16] introduced Who&When, a benchmark for automated failure attribution in multi-agent systems: 184 annotated failures across algorithm-generated and hand-crafted systems, each labeled with the responsible agent, the decisive step, and a natural language explanation. [1] introduced MAST, a multi-agent failure taxonomy with MAST-Data:

over 1,600 annotated traces across seven multi-agent frameworks, including a human-annotated subset (inter-annotator κ=0.88) and an LLM-as-judge annotation pipeline. MAST is the closest existing multi-agent failure dataset to Who&When. We scope this paper to AgentErrorBench and Who&When, the two datasets whose per-trajectory failure labels were available in mappable form when our prospective plan was frozen, and leave cross-walking MAST's 14-mode taxonomy into the unified scheme to future work; its human-annotated subset would be a natural validation set. For this audit, we use an analyst-defined 13-class coding scheme as a common vocabulary. Its operational definitions appear in Table 2; the native-to-unified mappings and retrospective coding corrections are released with the supplement. We do not claim that this scheme is a validated universal taxonomy. Our contribution is the explicit harmonization and quality audit of independently produced public annotations.

2 Materials & Methods

Our analysis plan, hypotheses and mapping rules were fixed before confirmatory statistics were computed (prospective analysis plan: `PREREGISTRATION.md` in the supplement). The per-model analysis in Section 3.5 is exploratory and was not prespecified; we label it as such.

2.1 Datasets

AgentErrorBench [18]: 200 annotated failure trajectories: 100 ALFWorld (embodied), 50 GAIA (general assistant), 50 WebShop (web navigation), produced by three models (GPT-4o: 81, Qwen3-8B: 62, Llama3.3-70B-Turbo: 57). Each trajectory carries one step annotation: a module key and a `failure_type` drawn from the AgentErrorTaxonomy, plus a free-text reasoning string. (The taxonomy's documentation is itself inconsistent: the README claims 17 error types, while the released definition file contains 18 distinct label strings across 19 module-qualified pairs; see Section 3.3.) We use the released label files (`alfworld_labels.json`, `gaia_labels.json`, `webshop_labels.json`) verbatim.

Who&When [16]: 184 annotated multi-agent failures (126 algorithm-generated, 58 hand-crafted systems such as Magnetic-One). Each record gives the responsible agent (`mistake_agent`), the decisive step, and a free-text `mistake_reason`. Only 4 of 184 records carry a structured `mistake_type`; the rest are free text. The 184 rows cover 139 unique questions; 45 questions contribute a second row.

The API-Bank dialogue corpus [2] was examined as a third candidate source but excluded: it contains API documentation and simulated exception strings but no failure annotations, so it cannot be mapped to the unified scheme without inventing labels. It is discussed qualitatively only.

2.2 Unified scheme and mapping

Representative native mappings are shown in Table 1. We map both datasets to our 13-class unified scheme (defined in Table 2): `PLAN`, `TOOL_SELECT`, `TOOL_FORMAT`, `TOOL_EXEC`, `OBS_MISREAD`, `GROUNDING`, `NO_RECOVERY`, `LOOP`, `GIVE_UP`, `VERIFICATION`, `MEMORY`, `COORDINATION`, `OTHER`. AgentErrorBench's native types map deterministically via published

Table 1: Representative native-to-unified mappings (full table released).

Native label	Unified	Basis
planning.inefficient_plan	PLAN	direct
planning.constraint_ignorance	PLAN	direct
reflection.progress_misjudge	OBS_MISREAD	direct
reflection.causal_misattribution	OBS_MISREAD	direct
memory.over_simplification	MEMORY	direct
reflection.hallucination	MEMORY	definition-based*
action.parameter_error	TOOL_FORMAT	direct
system.tool_execution_error	TOOL_EXEC	direct
system.step_limit	OTHER	scheme gap
action.misalignment	TOOL_SELECT	direct

*Module says reflection, but the published definition ("believes it performed actions that never occurred") describes false recall, a memory phenomenon.

definitions. System-level resource exhaustion (step_limit) has no unified counterpart and maps to OTHER, exposing a scheme gap we report rather than hide. Who&When's free-text reasons were coded by the author into the same scheme under a documented coding guide (e.g., "code is incorrect" $\rightarrow$ UNMAPPABLE (no malformed argument, schema violation, or unparseable call stated); "fabricated the population figures" $\rightarrow$ GROUNDING; "did not verify ... assumptions" $\rightarrow$ VERIFICATION). To measure coding reliability, the second author independently blind-coded all 184 Who&When rows against the original codebook (no access to the first coder's labels). Inter-annotator agreement was 69.6% (128/184), Cohen's $\kappa = 0.662$ (substantial). Disagreements concentrated on 34 rows where the second coder judged vague reasons unmappable (UNMAPPABLE), 27 of them bare "code is incorrect" reasons the coding guide had mapped to TOOL_FORMAT. Applying the Table 2 definition of TOOL_FORMAT (malformed arguments, schema violations, unparseable calls), those 27 rows are coded UNMAPPABLE in the consensus (Section 3.6). All 56 disagreements (30% of rows) were adjudicated to consensus labels; in the final consensus, 4 resolve to the first coder's label, 51 to the second's, 1 to a third reading; all analyses use these retrospectively corrected labels. The original consensus labels and notes are preserved alongside the revised labels; this correction is not a new independent human annotation round. Coder-specific distributions and sensitivity analyses of the contested mappings are reported in Section 3.6, which shows which findings depend on the disputed decisions and which survive them. All 184 codings with rationales are released. Records that cannot be mapped without guessing are coded UNMAPPABLE and reported, never silently dropped.

2.3 Statistics

Per-dataset proportions use 95% Wilson confidence intervals. H1, the domain comparison (RQ4) and the exploratory model comparison use χ^2 tests of independence (α=0.05) with standardized residuals; effect size via Cramer's V. Because several expected cell counts fall below five, we additionally computed Monte Carlo permutation p-values (100,000 uniform label permutations

per test, seed 7, Pearson χ^2 statistic, group sizes fixed). Exceedance counts: H1 $0/100{,}000$; domain $0/100{,}000$; model $195/100{,}000$ (p=0.0020). Conclusions were unchanged at α=0.05. (An earlier version of the permutation script contained a vectorization bug that pooled permutations before tabulating, understating p; the corrected implementation was verified against an independent plain-loop reimplementation, and we report the corrected figures throughout.) Residuals are used descriptively to localize each omnibus association; individual cells are not treated as separately confirmatory tests. H2 compares the plurality class per domain; H3 is a one-sided proportion check. Deviation from the prospective plan: the plan listed Jensen-Shannon divergence between conditional sub-label distributions as part of the RQ3 label-shift audit; it was not computed because the datasets do not provide comparable shared sub-label distributions, so RQ3 was evaluated through definition comparison and the label-quality audit instead. Second deviation: the frozen coding guide mapped bare "code is incorrect" reasons to `TOOL_FORMAT`; applying the Table 2 class definition, 27 such rows were recoded `UNMAPPABLE` (Section 3.6). The hypotheses and confirmatory tests are unchanged.

3 Results

We report the confirmatory results for RQ1–RQ4 and the prespecified hypotheses H1–H3, followed by the exploratory model comparison (Section 3.5), robustness and sensitivity analyses (Section 3.6), and domain narratives from the residuals (Section 3.8).

3.1 RQ1: Within-dataset distributions

AgentErrorBench (n=200) is dominated by `PLAN` (69; 34.5%, 95% CI 28.3–41.3%), followed by `OBS_MISREAD` (37; 18.5%) and `MEMORY` (35; 17.5%). `TOOL_EXEC` (6.0%), `OTHER` (4.0%), `TOOL_SELECT` (3.5%) and `TOOL_FORMAT` (2.5%) are rare; 27 records (13.5%, CI 9.4–18.9%) are `UNMAPPABLE`. Who&When (n=184) shows a markedly different profile: among mapped classes, `GROUNDING` (36; 19.6%, CI 14.5–25.9%) leads, then `VERIFICATION` (28; 15.2%, CI 10.7–21.1%), `PLAN` (20; 10.9%, CI 7.1–16.2%), `TOOL_EXEC` (15; 8.2%), `TOOL_FORMAT` (10; 5.4%, CI 3.0–9.7%) and `OTHER` (10; 5.4%), `TOOL_SELECT` (8; 4.3%), `OBS_MISREAD` (7; 3.8%), `COORDINATION` (6; 3.3%), `LOOP` (5; 2.7%), `GIVE_UP` (3; 1.6%) and `MEMORY` (1; 0.5%). A further 35 records (19.0%, CI 14.0–25.3%) are `UNMAPPABLE`. Notably, no released annotation in either dataset mapped to `NO_RECOVERY`, and `MEMORY` is prevalent in AgentErrorBench but appears only once in Who&When, while `VERIFICATION`, `GROUNDING`, `LOOP`, `GIVE_UP` and `COORDINATION` appear only in Who&When (Fig. 1).

3.2 RQ2/H1: Cross-dataset comparison

The distributions differ overwhelmingly: χ^2=160.48, df=12, p=4.2×10^{-28}, Cramer's V=0.646 (permutation $p\approx10^{-5}$, $0/100{,}000$ exceedances; **H1 supported**). Standardized residuals ($|r|>2$) show AgentErrorBench over-representing `PLAN` ($r=+3.33$), `OBS_MISREAD` (+2.94) and `MEMORY` (+3.75), while Who&When over-represents `GROUNDING` (+4.51) and `VERIFICATION` (+3.98). Excluding `UNMAPPABLE` as a robustness check leaves the conclusion unchanged (χ^2=158.93, df=11, p=2.2×10^{-28}, Cramer's V=0.703, a very large effect). In plain terms: the

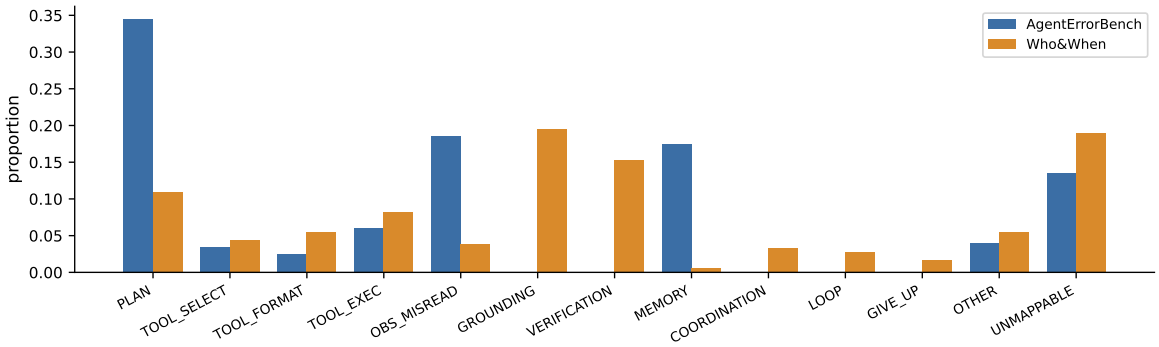


Fig. 1: Unified failure-class distribution by dataset. Percentages use all released rows within each dataset.

observed distributions are strongly associated with dataset membership, so aggregate failure frequencies should not be generalized across these datasets.

3.3 RQ3: Do labels mean the same thing? Label-noise audit

Two findings. First, a qualitative comparison of AgentErrorTaxonomy's published definitions against our unified classes reveals genuine incommensurability at the margins: e.g., reflection-module `hallucination` ("believes it performed actions that never occurred") is a memory phenomenon wearing a reflection label (Table 1), and `step_limit` (resource exhaustion) has no counterpart in the unified scheme at all.

Second, the released AgentErrorBench labels are noisy *in themselves*: 27 of 200 annotations (13.5%) carry an **empty** `failure_type` with empty reasoning, unmappable without guessing. The empties are not random: 24 of 57 Llama3.3-70B-Turbo trajectories (42.1%) are empty versus 3 of 62 Qwen3-8B (4.8%) and 0 of 81 GPT-4o (0%; see Fig. 2), suggesting a systematic annotation-pipeline gap for one model. The files additionally contain inconsistent category strings requiring normalization before any analysis (Fig. 3 shows the raw label distribution): `plan_inefficient` vs. `inefficient_plan`, `Parameter_error` vs. `parameter_error`, a trailing-space variant of `tool_execution_error`, and a `plan` vs. `planning` module-key split between WebShop and GAIA files. Even the taxonomy's size is inconsistently documented: the README claims 17 error types, but `error_definitions.py` defines 18 distinct label strings (19 module-qualified pairs). Any secondary analysis that naively counts native labels will therefore undercount or split categories. Table 3 lists every normalization our curation applied; we recommend benchmark releases ship such a table themselves rather than leaving it to secondary analysts to discover.

3.4 RQ4/H2/H3: Domains and hypotheses

Failure profiles differ significantly by domain (χ^2=185.29, df=33, p=4.1×10^{-23}, permutation $p \approx 10^{-5}$ (0/100,000 exceedances)). This test excludes the 62 `UNMAPPABLE` records and the all-zero `NO_RECOVERY` class. Plurality classes: WebShop `PLAN` (19/49, 38.8%), ALFWorld `PLAN` (24/74, 32.4%), GAIA `PLAN` (26/50, 52.0%), multi-agent `GROUNDING` (36/184 records overall,

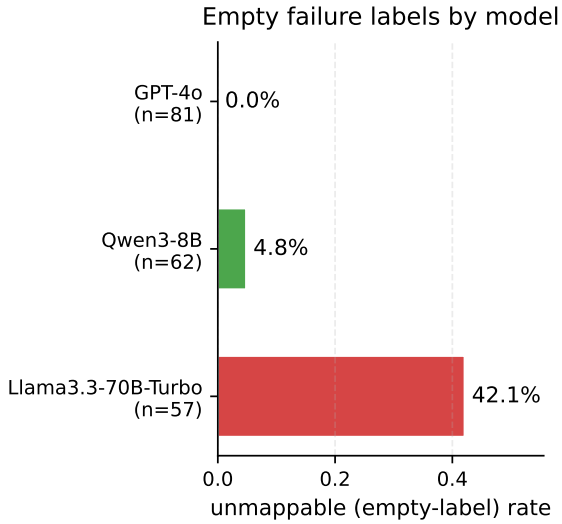


Fig. 2: Unmappable (empty) label rate by model in AgentErrorBench. The noise is concentrated in one model's trajectories.

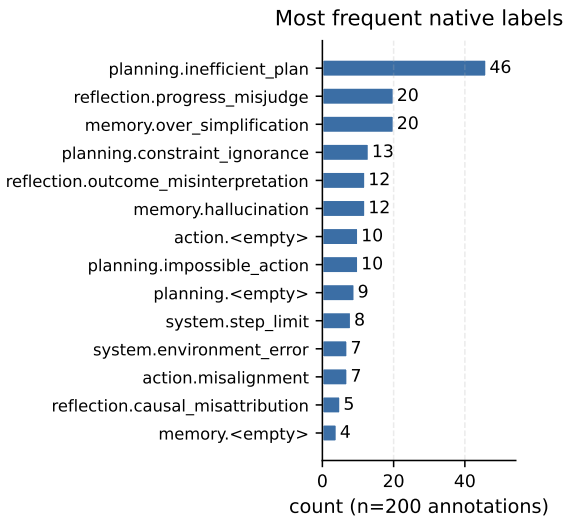


Fig. 3: Most frequent native failure labels in AgentErrorBench after label-string normalization, before unified mapping (n=200).

19.6%; 36/149 mapped records, 24.2%). **H2 is not supported**: while planning failures do form the plurality in web navigation as predicted, coordination failures do *not* dominate the multi-agent data (3.3%, ninth of twelve mapped classes). Multi-agent failures in Who&When are instead dominated by fabricated information and unchecked outputs: individual-agent competence failures, not interaction failures. A further 35/184 (19.0%) multi-agent reasons are `UNMAPPABLE` under the corrected coding rule (Section 3.6). **H3 is not supported**: an exact one-sided binomial test of H_0: $p \leq 0.15$ against H_1: $p > 0.15$ on the 27/200 empty native labels gives p=0.75. (H3 uses AgentErrorBench alone: its `UNMAPPABLE` records are empty native failure_type fields, a property of the released labels, whereas Who&When carries no native labels at all, so the construct is not comparable across datasets.) Table 4 gives the full contingency table, shown visually in Fig. 4.

3.5 Exploratory: failure profiles differ by model

Within AgentErrorBench, the three evaluated models show significantly different failure profiles (χ^2=30.72, df=12, p=0.002, permutation p=0.0020 (195/100,000 exceedances), Cramer's V=0.30, excluding unmappable records). GPT-4o's labeled failures concentrate in `PLAN` (29/81), `MEMORY` (23/81) and `OBS_MISREAD` (17/81); Qwen3-8B is strikingly plan-heavy (`PLAN` 30/59 labeled, 50.8%) with `MEMORY` significantly under-represented ($r = -2.88$); Llama3.3-70B-Turbo's 33 labeled failures spread across `PLAN` (10), `MEMORY` (10) and `TOOL_EXEC` (5). Models differ in domain composition, so the pooled comparison could in principle reflect domain mix rather than model: GPT-4o's trajectories are 50/81 ALFWorld, versus 23/57 for Llama3.3-70B-Turbo and 27/62 for Qwen3-8B. The model effect survives holding domain fixed: permuting model labels within domains leaves the pooled association significant (stratified permutation p=0.0032, 20,000 permutations), and within the single largest shared domain, ALFWorld (n=74 labeled), GPT-4o and Qwen3-8B still differ sharply (χ^2=21.23, df=6, p=0.0017, Cramer's V=0.54;

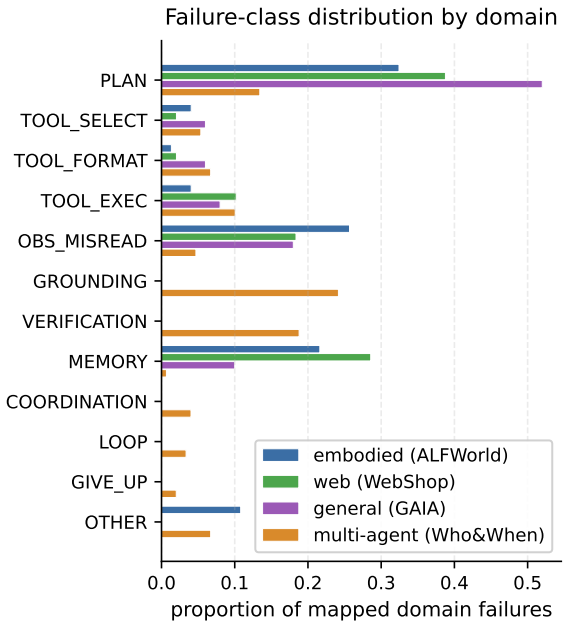


Fig. 4: Unified failure-class distribution by domain among mapped records (embodied, web, general-assistant, multi-agent).

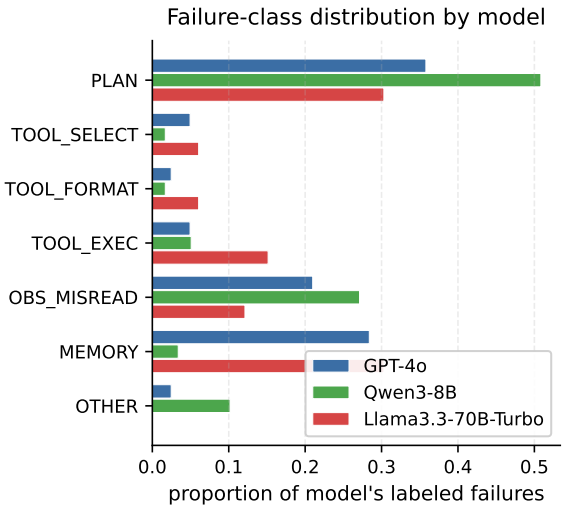


Fig. 5: Unified failure-class distribution by model (AgentErrorBench, labeled records only).

Qwen records no `MEMORY` failures in ALFWorld, 0/24, versus 16/50 for GPT-4o). Llama3.3-70B-Turbo contributes no labeled ALFWorld trajectories at all (all 23 empty), so it drops out of that within-domain check. In other words, among available labeled trajectories, failure profiles differed across models: even holding the benchmark fixed, *which model* failed predicts *how* it failed. This comparison is distorted by non-random missingness, however: with 42.1% of Llama3.3-70B-Turbo's annotations empty versus 0% for GPT-4o, the labeled Llama trajectories may not represent that model's full failure profile, so the model-level result should be read as exploratory. The observed frequencies vary across these models (Fig. 5), but this does not establish a failure mechanism or taxonomy measurement invariance across models.

3.6 Robustness: coder-specific distributions and sensitivity analyses

The Who&When coding hinged on one consequential boundary decision: the original coding guide mapped bare "code is incorrect" reasons to `TOOL_FORMAT` (produced artifact malformed), while the second coder judged 34 explanations too vague to map at all (`UNMAPPABLE`). Of the 56 independent disagreements, 27 were coder-1-`TOOL_FORMAT` versus coder-2-`UNMAPPABLE`. The two coders' distributions differ accordingly: coder 1: `TOOL_FORMAT` 40 (21.7%), `GROUNDING` 38 (20.7%), `PLAN` 23 (12.5%); coder 2: `GROUNDING` 37 (20.1%), `UNMAPPABLE` 34 (18.5%), `VERIFICATION` 29 (15.8%), `TOOL_FORMAT` 9 (4.9%), but agree on the classes they *don't* dispute: `GROUNDING`, `VERIFICATION` and `PLAN` appear at similar ranks in both.
We retrospectively revised the boundary by applying the Table 2 definition of `TOOL_FORMAT` as written (malformed arguments, schema violations, unparseable calls): a reason is coded

TOOL_FORMAT only if it describes a defect in the form of a call or its arguments. Bare "code is incorrect" reasons, and reasons describing logic defects or outcomes (wrong results, no output, unhandled edge cases) without a defect in the call's form, are coded UNMAPPABLE. This recodes 27 consensus rows from TOOL_FORMAT to UNMAPPABLE (listed with rationales in the released coding table); the recoded set coincides exactly with the 27 coder-1-TOOL_FORMAT versus coder-2-UNMAPPABLE disagreements, not with all 34 second-coder unmappable rows. Ten retained TOOL_FORMAT labels use a broader operational interpretation, including placeholder values, missing imports, nonexistent columns and inadequate search queries. Some describe semantic or implementation problems rather than a demonstrated schema or parsing failure. We therefore also report a strict sensitivity analysis that recodes all ten as UNMAPPABLE; the dataset association persists (χ^2=167.29, df=12, p=1.7×10^{-29}, V=0.660). Grounding remains the largest mapped class, while the unmappable share rises to 45/184 (24.5%). This exposes a remaining construct boundary rather than resolving it by assumption. The consensus Who&When distribution is now GROUNDING 36 (19.6%), VERIFICATION 28 (15.2%), PLAN 20 (10.9%), TOOL_FORMAT 10 (5.4%). As a remaining sensitivity check, we re-ran the headline tests using all second-coder labels instead of the consensus: the omnibus H1 test is essentially unchanged (χ^2=157.41, df=12, p=1.8×10^{-27}, V=0.640, vs. consensus χ^2=160.48, V=0.646), as is the domain comparison (χ^2=183.15 vs. 185.29), and the Who&When top class remains GROUNDING (37/184). The cross-dataset distributional gap and the grounding plurality are therefore robust to the remaining coder disagreement; the earlier tool-formatting plurality is absent from the revised consensus and coder 2, although it remains present in coder 1.

3.7 Cluster-aware sensitivity: question-level dependence

Who&When contributes 184 rows from 139 unique questions, with 45 questions contributing two rows. We added a retrospective sensitivity analysis that keeps each question intact.
AgentErrorBench's 200 released trajectory IDs are unique; shared underlying task IDs cannot be established from these labels, so residual dependence between its trajectories remains possible.
For the dataset comparison there are 339 clusters; after removing unmappable rows the domain comparison has 286 nonempty clusters.
We permuted group labels within cluster-size strata, preserving the number of clusters per group and the observed multiplicities. Two-row questions occur only in Who&When and therefore do not exchange dataset labels with singleton trajectories. Using Pearson's statistic, seed 7 and 20,000 permutations, both associations have zero exceedances; the plus-one Monte Carlo estimate is p=$1/20{,}001 \approx 5 \times 10^{-5}$. These conditional reference tests assume exchangeability within size strata and are not causal tests of taxonomy effects.
Resampling the 139 Who&When questions with replacement 20,000 times (seed 7) yields row-weighted grounding 19.6% (percentile 95% CI 14.1–25.3%) and coordination 3.3% (CI 1.1–6.0%). Grounding is a plurality among mapped classes in 86.7% of samples when ties count, and the unique plurality in 83.5%. These intervals capture empirical question-level sampling variation, not coding uncertainty or population representativeness. The full procedure and outputs are released.

3.8 Domain narratives: what the residuals say

Standardized residuals by domain (Fig. 6) turn the contingency table into four interpretable stories. *Embodied (ALFWorld)* failures over-represent observation misreading ($r = +2.8$) and memory lapses ($+2.7$) while under-representing grounding (-2.9) and verification (-2.5): the released annotations emphasize observation interpretation and memory. These frequencies do not independently establish causal mechanisms. *General-assistant (GAIA)* failures are plan-dominated ($+3.3$) with grounding (-2.4) and verification (-2.1) under-represented: the released general-assistant annotations emphasize planning. *Web (WebShop)* failures over-represent memory ($+3.6$) while under-representing grounding (-2.3) and verification (-2.1): the released web annotations emphasize memory and constraints; we did not experimentally compare debugging interventions. *Multi-agent (Who&When)* is the mirror image: grounding ($+4.7$) and verification ($+4.2$) are over-represented while memory (-3.8), planning (-3.3) and observation misreading (-3.0) are under-represented. Strikingly, `COORDINATION` never rises above a minority class in any domain; under the consensus coding, the multi-agent annotations emphasize individual-agent content and verification problems more often than explicit coordination labels. Each domain's profile is coherent on its own terms; none of them is the universal profile.

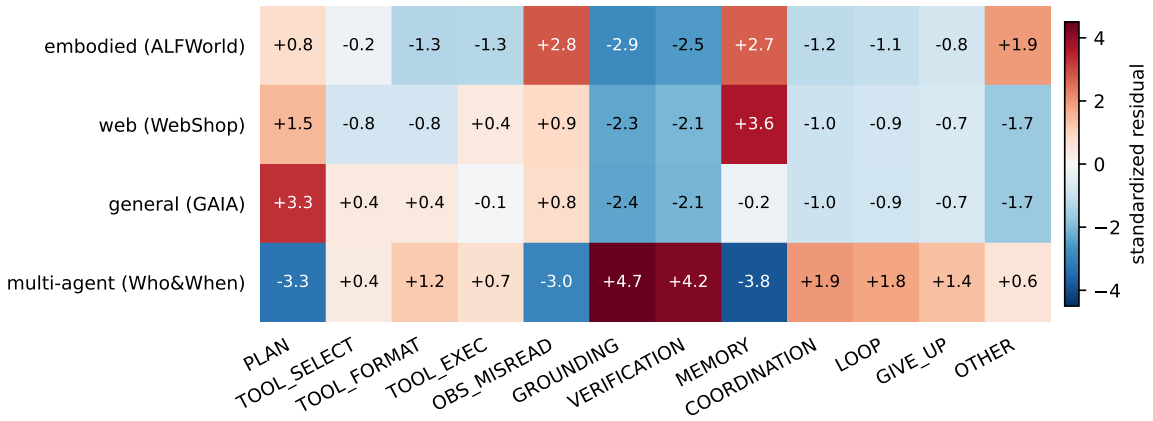


Fig. 6: Standardized residuals of unified failure class by domain. Positive cells are over-represented relative to independence. Residuals are used descriptively to localize the omnibus association; individual cells are not interpreted as separately confirmatory tests.

4 Discussion

Five implications follow. First, **harmonized failure-label distributions differ dramatically across the public datasets studied here**, but we cannot attribute the gap to the taxonomies alone. The two datasets differ simultaneously in tasks, systems, and annotation protocols, which co-vary with the label differences, so a distributional gap cannot isolate a taxonomy effect. This audit shows that aggregate frequencies depend on benchmark context and coding choices. Frequency differences alone neither prove nor disprove that a category system can be applied consistently to

new trajectories. Cross-paper claims of the form "planning remains the dominant failure mode" are at minimum benchmark-, domain- and model-relative, not universal. This cautions against a growing practice in the evaluation literature [15]: treating taxonomy categories as portable measurement instruments.

Second, **benchmark label quality is a first-order problem**. A 13.5% empty-label rate concentrated in one model's trajectories, inconsistent category strings, and documentation mismatches mean published failure-profile numbers built on these files should be treated with caution until the annotations are repaired. The field currently invests heavily in taxonomy *design* [18, 16]; our audit suggests equal investment is needed in annotation *execution and quality control*.

Third, the **multi-agent surprise**: if coordination failures are a characteristic failure mode of multi-agent systems, we might expect them to be prominent in a benchmark explicitly studying multi-agent failures; instead, grounding failures dominate here (36/184, 19.6%); tool-formatting failures are uncommon (10/184) once bare "code is incorrect" reasons are coded as unmappable per the class definition (Section 3.6). This could reflect annotation focus or genuine task differences; the present data do not distinguish those explanations.

Fourth, no released annotation in either dataset mapped to `NO_RECOVERY` under our coding procedure, which suggests the field's vocabulary for "failure to recover" outruns its measurement of it. Recovery behavior may be the most under-instrumented dimension of agent evaluation, and benchmarks in the ReAct tradition [14] are well placed to instrument it: future reason–act trajectory releases could explicitly label whether recovery was attempted and how it failed.

Fifth, **for benchmark designers**, our audit yields concrete recommendations : (1) validate annotation pipelines per model, not just per benchmark: our empty labels clustered in one model's trajectories; (2) enforce a controlled vocabulary for category strings at write time (case, spacing and naming variants silently split categories downstream); (3) keep documentation and definition files in sync (the 17-vs-18 mismatch is a one-line fix); (4) publish inter-annotator agreement for failure labels, as is standard for the task labels themselves; and (5) annotate *recovery attempts*, the dimension our data suggest is most neglected.

4.1 Guidance for agent builders

Our results also carry practical lessons for practitioners choosing where to harden agents. *Match the mitigation to the domain, not to the literature average.* An embodied agent's failure budget is dominated by observation misreading and memory ($r = +2.8$, $+2.7$ in ALFWorld); invest in perception checks and episodic state tracking before fancier planning. A web-shopping agent fails on memory of constraints ($+3.6$ in WebShop), so constraint-carrying context deserves the engineering effort. A general-assistant agent fails at planning ($+3.3$ in GAIA), which is where decomposition and verification scaffolds are the natural hypothesis to test first. *Do not assume multi-agent means coordination risk:* in the multi-agent dataset studied here, coordination was 3.3% of consensus-coded failures while grounding failures dominated, observation handling and individual tool use are candidate interventions to evaluate before investing primarily in coordination protocols; 19.0% of the multi-agent reasons are unmappable under the corrected coding (Section 3.6). *Expect model-specific failure signatures:* Qwen3-8B's plan-heavy profile versus GPT-4o's memory-heavy one means a mitigation tuned on one model may not transfer to another; evaluate failure profiles per model you ship. Finally, *instrument recovery explicitly*: no

released annotation in either dataset mapped to `NO_RECOVERY`, so recovery behavior is unmeasured in these datasets, and methods like Reflexion [7] suggest structured recovery is a candidate for evaluation rather than an intervention tested here. If your evaluation logs trajectories, label the recovery attempts; these datasets leave you flying blind on them.

4.2 Threats to validity

Construct validity: our unified scheme inherits boundary decisions in the analyst-defined scheme; a different scheme could shift marginal cases (e.g., `reflection.hallucination`). We mitigate this by publishing the full mapping with rationales and reporting scheme gaps explicitly. *Internal validity:* Who&When codings were produced under a documented guide and independently second-coded (agreement 69.6%, $\kappa = 0.662$, substantial); all 56 disagreements were adjudicated to consensus labels, and Section 3.6 reports coder-specific distributions plus sensitivity analyses showing which findings survive alternative resolutions. Residual risk is that both coders share the guide's boundary assumptions, so the per-model analysis stays exploratory rather than confirmatory. *External validity:* 384 failures from two datasets cannot represent the full space of agent failures; both datasets are English-language and researcher-curated, software-engineering agents of the SWE-agent type [11] are unrepresented, and the recent MAST dataset [1] is not covered here. Moreover, the 184 Who&When rows contain only 139 unique question IDs (45 questions contribute a second row each), so we re-ran the confirmatory tests with question-level units (Section 3.7): the associations remain significant under cluster permutation, and the grounding plurality holds in 86.7% of cluster-bootstrap resamples when ties count. *Conclusion validity:* the original hypotheses and row-level tests were prespecified; the revised coding, cluster checks and model stratification are retrospective sensitivity analyses. The exploratory model comparison is clearly marked and should be confirmed on larger, balanced samples.

4.3 Limitations

Beyond the threats above: the sample covers three models in only one dataset; AgentErrorBench's empty labels reduce the effective Llama sample to 33; and our domain comparison conflates benchmark, task distribution and annotation protocol, which co-vary in observational data. We therefore frame the study as a descriptive audit of harmonized failure annotations, not a controlled test of taxonomy portability: the observed gaps cannot isolate a taxonomy effect, and the observed frequencies should not be treated as universal rates. A controlled experiment (same tasks, same annotators, varying only the taxonomy) would isolate the taxonomy effect, and we encourage it.

5 Conclusions

We audited whether agent failure taxonomies travel across datasets, by harmonizing 384 annotated failures from two independently produced public datasets into one 13-class scheme. The harmonized label distributions differ dramatically across datasets, domains and models; native labels contain nontrivial noise; and key characterizations (notably the multi-agent grounding plurality and the 19.0% unmappable share) are robust to the remaining coding sensitivity (Section 3.6). Because tasks, systems, and annotation protocols co-vary across the

datasets, these gaps cannot isolate a taxonomy effect, and the observed frequencies should not be treated as universal rates. The field should report the benchmark context of failure-label frequencies and invest in label quality with the same seriousness it invests in taxonomy design. For practitioners, the domain narratives in Section 3.8 give a concrete starting point, framed as hypotheses to test rather than tested prescriptions: harden perception and memory for embodied agents, constraint tracking for web agents, planning for assistant agents, and individual tool use for multi-agent systems. For benchmark designers, the five recommendations in Section 4 are cheap to implement and would make the next generation of failure datasets genuinely comparable. Until then, any claim about "how agents fail" should come with a benchmark name attached.

6 Data Availability

The annotations for 384 failures, native-to-unified mappings, original and revised Who&When labels, the 27-row recoding audit, the prospective analysis plan, all analysis and resampling scripts, generated results and figure sources are provided in Supplemental Data S1. The matching public repository is `https://github.com/abhinav-TB/agent-failure-taxonomies-travel`. Original analysis artifacts are released under the MIT license; upstream materials retain their original terms as documented in the supplement.

Funding

The authors received no funding for this work.

Competing Interests

The authors declare no competing interests.

References

[1] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. *Why do multi-agent LLM systems fail?* 2025. arXiv: 2503.13657. URL: https://arxiv.org/abs/2503.13657.

[2] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li. "API-Bank: A comprehensive benchmark for tool-augmented LLMs". In: *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*. 2023.

[3] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. *AgentBench: Evaluating LLMs as agents*. 2023. arXiv: 2308.03688. URL: https://arxiv.org/abs/2308.03688.

[4] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. “GAIA: A benchmark for general AI assistants”. In: *Proc. Int. Conf. Learn. Representations (ICLR)*. 2024.

[5] J. Ruan, Y. Chen, B. Zhang, Z. Xu, T. Bao, G. Du, S. Shi, H. Mao, Z. Li, X. Zeng, and R. Zhao. “TPTU: Large Language Model-based AI Agents for Task Planning and Tool Usage”. In: *NeurIPS 2023 Workshop on Foundation Models for Decision Making*. *arXiv:2308.03427*. 2023.

[6] R. K. Sharma, S. Barke, and B. Zorn. *Willful disobedience: Automatically detecting failures in agentic traces*. 2026. arXiv: 2603.23806. URL: https://arxiv.org/abs/2603.23806.

[7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. “Reflexion: Language agents with verbal reinforcement learning”. In: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*. Vol. 36. *arXiv:2303.11366*. 2023. DOI: 10.52202/075280-0377.

[8] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox. “ALFRED: A benchmark for interpreting grounded instructions for everyday tasks”. In: *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2020, pp. 10740–10749.

[9] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht. “ALFWorld: Aligning text and embodied environments for interactive learning”. In: *Proc. Int. Conf. Learn. Representations (ICLR)*. 2021.

[10] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su. “TravelPlanner: A benchmark for real-world planning with language agents”. In: *Proc. Int. Conf. Mach. Learn. (ICML)*. Vol. 235. *arXiv:2402.01622*. 2024, pp. 54590–54613. URL: https://proceedings.mlr.press/v235/xie24j.html.

[11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. “SWE-agent: Agent-computer interfaces enable automated software engineering”. In: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*. 2024.

[12] S. Yao, H. Chen, J. Yang, and K. Narasimhan. “WebShop: Towards scalable real-world web interaction with grounded language agents”. In: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*. 2022.

[13] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. “τ-bench: A benchmark for tool-agent-user interaction in real-world domains”. In: *Proc. Int. Conf. Learn. Representations (ICLR)*. 2025.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. “ReAct: Synergizing reasoning and acting in language models”. In: *Proc. Int. Conf. Learn. Representations (ICLR)*. 2023.

[15] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer. *Survey on evaluation of LLM-based agents*. 2025. arXiv: 2503.16416. URL: https://arxiv.org/abs/2503.16416.

[16] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu. *Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems*. 2025. arXiv: 2505.00212. URL: https://arxiv.org/abs/2505.00212.

[17] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. "WebArena: A realistic web environment for building autonomous agents". In: *Proc. Int. Conf. Learn. Representations (ICLR)*. 2024.

[18] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You. *Where LLM agents fail and how they can learn from failures*. 2025. arXiv: 2509.25370. URL: https://arxiv.org/abs/2509.25370.

Table 2: The 13 unified failure classes (our frozen scheme), with definitions and illustrative examples. No released annotation mapped to NO_RECOVERY under our coding procedure.

Class	Definition	Example
PLAN	Wrong decomposition, incoherent strategy, goal misunderstanding	agent pursues steps in an impossible order
TOOL_SELECT	Wrong tool/API chosen for a reasonable sub-goal	web search called for arithmetic needing a calculator
TOOL_FORMAT	Malformed arguments, schema violations, unparseable calls	JSON missing a required field; wrong parameter type
TOOL_EXEC	The tool itself fails: crashes, timeouts, auth failures	API timeout; environment crashes mid-episode
OBS_MISREAD	Correct observation, wrong inference drawn from it	"item out of stock" read as purchase success
GROUNDING	Hallucinated or unsupported observations cited as evidence	quotes a price appearing nowhere on the page
NO_RECOVERY	Informative error followed by verbatim retry or abandonment without trying an alternative	resubmits the identical call after "invalid date format"
LOOP	Repeated near-identical actions with no forward progress	same search query issued five times in a row
GIVE_UP	Premature termination; false infeasibility claim	declares the task impossible after one failed attempt
VERIFICATION	Skipped or incorrect verification before acting/answering	final answer submitted without checking it
MEMORY	False recall, omission, forgetting, history errors	forgets a constraint stated three turns earlier
COORDINATION	Multi-agent only: role disobedience, inter-agent misalignment, false consensus	executor ignores the planner's revised plan
OTHER	Residual: named category fitting none of the above	disallowed content; corrupt task inputs

Table 3: Label-string normalizations applied during curation. The first three variants were attested in the repository's definition files; the module-key split was attested in the released data files. In the data files the normalized planning.inefficient_plan form accounts for 46 of 200 records.

Normalization	Attested in
plan_inefficient → inefficient_plan	definition files
Parameter_error → parameter_error	definition files
trailing space after tool_execution_error	definition files
plan.* → planning.* (module key)	data files

Table 4: Unified failure-class counts by domain (row: class; columns: embodied, web, general-assistant, multi-agent).

Class	ALFWorld (n=100)	WebShop (n=50)	GAIA (n=50)	Multi-agent (n=184)
PLAN	24	19	26	20
OBS_MISREAD	19	9	9	7
MEMORY	16	14	5	1
TOOL_EXEC	3	5	4	15
TOOL_FORMAT	1	1	3	10
TOOL_SELECT	3	1	3	8
GROUNDING	0	0	0	36
VERIFICATION	0	0	0	28
COORDINATION	0	0	0	6
OTHER	8	0	0	10
GIVE_UP	0	0	0	3
LOOP	0	0	0	5
NO_RECOVERY	0	0	0	0
UNMAPPABLE	26	1	0	35